

CSDS119: Computer Vision and Image Processing

LAB 1: Understanding Digital Images Using Python

Platform: Google Colab / Jupyter Notebook

Tasks:

- Read an image in Python.
- Understand that an image is a NumPy matrix.
- Identify image dimensions, channels and bit depth.
- Access and modify pixels.
- Perform basic intensity transformations.
- Visualize histograms and binary images.
- Understand the effect of resizing and noise.
- Apply basic mean and median filtering.

Part 1 — Setup

Run the following cell to import the required libraries.

```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt

print('Libraries imported successfully!')
```

Libraries imported successfully!

Part 2 — Upload and Read an Image

The next cell allows you to upload an image directly in Google Colab. In Jupyter, you may instead place an image in the same folder as this notebook.

```
In [2]: # Works directly in Google Colab
try:
    from google.colab import files
    uploaded = files.upload()
    filename = next(iter(uploaded))
except ImportError:
    filename = input('Enter image filename (e.g., sample.jpg): ')

img = cv2.imread(filename)

if img is None:
    raise FileNotFoundError('Image could not be read. Check the filename/')
```

```
plt.figure(figsize=(7,5))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title('Original Image')
plt.axis('off')
plt.show()
```

Original Image



Think Before Moving Ahead

Question: Is Python storing this image as a photograph or as numbers?

Part 3 — Explore Image Properties

```
In [3]: print('Data type:', type(img))
        print('Shape:', img.shape)
        print('Data type of pixels:', img.dtype)
        print('Minimum pixel value:', img.min())
        print('Maximum pixel value:', img.max())
```

```
Data type: <class 'numpy.ndarray'>
Shape: (451, 680, 3)
Data type of pixels: uint8
Minimum pixel value: 0
Maximum pixel value: 255
```

Understand the Output

For a color image with shape `(height, width, 3)` :

- Height = number of rows
- Width = number of columns
- 3 = color channels

- `uint8` means each channel is represented using 8 bits

For an 8-bit image: $L=2^8=256$ so the intensity range is normally 0 to 255.

Think: Why can't a `uint8` image store the value 300?

Part 4 — Convert the Image to Grayscale

```
In [4]: gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title('Color Image')
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(gray, cmap='gray')
plt.title('Grayscale Image')
plt.axis('off')

plt.tight_layout()
plt.show()

print('Color shape:', img.shape)
print('Grayscale shape:', gray.shape)
```

Color Image



Grayscale Image



```
Color shape: (451, 680, 3)
Grayscale shape: (451, 680)
```

Think: Why did the third dimension disappear after converting to grayscale?

Part 5 — See the Image as a Matrix

A digital grayscale image is a matrix of intensity values.

```
In [1]: #add your code here
```

Pixel Access

Remember: NumPy uses `[row, column]`, i.e. `[y, x]`.

```
In [2]: #add your code here
```

Part 6 — Modify One Pixel

The following experiment changes only one pixel to white.

```
In [3]: #add your code here
```

Think: Only one number changed. Why is the change difficult to notice in the complete image?

Part 7 — Intensity Transformation: Negative

For an 8-bit grayscale image: $s = 255 - r$ where r is the input intensity and s is the output intensity.

```
In [4]: #add your code here
```

Think: Why does black become white and white become black?

Part 8 — Brightness Transformation

Increase brightness by 50: $s = r + 50$ Because an 8-bit image cannot contain values above 255, clipping is required.

```
In [5]: #add your code here
```

Think: Why is `np.clip()` necessary here?

Part 9 — Thresholding

A simple threshold transforms a grayscale image into a binary image:
$$s(x,y) = \begin{cases} 255, & r(x,y) > T \\ 0, & r(x,y) \leq T \end{cases}$$

```
In [6]: #add your code here
```

Investigation

Change `T` to 50, 100, 180 and 220. Observe what happens.

Think: Why does changing the threshold change the visible object/region?

Part 10 — Histogram

In [7]: `#add your code here`

Think: Without looking at the image, what information can the histogram tell you about the distribution of pixel intensities?

Part 11 — Cropping and Resizing

In [8]: `#add your code here`

Think: Which operation changes the spatial dimensions? Does resizing necessarily create new image information?

Part 12 — Add Salt-and-Pepper Noise

The following creates impulse noise by randomly changing some pixels to 0 or 255.

In [9]: `#add your code here`

Part 13 — Mean vs Median Filtering

Now connect the experiment to order-statistic filtering.

In [10]: `#add your code here`

Critical Thinking

Compare the noisy image, mean-filtered image and median-filtered image.

1. Which filter removes the salt-and-pepper noise more effectively?
2. Which filter preserves edges better?
3. Why can an extreme value such as 255 strongly affect the mean but have much less effect on the median?
4. What do you expect if the window is changed from 3×3 to 7×7?

Part 14 — Manual Order-Statistic Exercise

Consider the following 3×3 neighborhood:

$$\begin{bmatrix} 42 & 43 & 44 \\ 43 & 255 & 45 \\ 44 & 45 & 46 \end{bmatrix}$$

Before running Python, calculate manually:

- Minimum
- Maximum

- Median
- Mean
- Midpoint

Then execute the following cell.

```
In [11]: #add your code here
```

```
In [ ]:
```